object itself. RBS lets you create *union types*, which are one type or another, with the | character. So a method that takes an ID or an object might have the signature `def name: (Integer | User) -> String`.

Any Ruby literal (including `nil`) can be used as a RBS type, such as in the unusual example where you want to limit a variable to a single value. That's rare, but you can use it to build up union types, such as the built-in type `bool`, which is defined as `true | false`. In Ruby, any object has a truth value, not just the boolean literals. In RBS, if you want to mark that you are taking in an arbitrary object to use it for its truth value, RBS provides the type `boolish`, as in `def name: (use_full: boolish) -> String`.

Because any Ruby object can have a truth value, any Ruby object can be of type `boolish`, it's more a marker for the developers that the value is only being used for its truth value. RBS has a couple of other defined ways to refer to a value that can be of any type. Generically, if you want to really specify that an object is known to have any type, you use `top`, which is a supertype of all types. If you are allowing any object, but only because you don't know any better about the type system, you should use `untyped` rather than top. And if you are signaling that a value shouldn't be used, as in a method whose return value is uninteresting, you should use `void`.

You can specify the type of each element of a fixed-length array using something like array syntax: `[String, String]`. Similarly, you can specify the type of a hash per key: `{name: String, age: Integer}`. Note that Ruby does not enforce these at run time; they are currently used only for static analysis and for hints when developing.

For methods, if the return value is overloaded, you can use union syntax to define multiple versions of the method at once:

```
def name: (Integer) -> String
  | (User) -> String
```

Somewhat unusually for Ruby, RBS allows you to define an *interface*, meaning a subset of methods that might be implemented by more than one class. By convention, interface names start with an underscore:

```
interface _Loggable
  def log: (?String) -> String
end
```

You can then use `_Loggable` as an RBS type, and it will assume nothing about the underlying object, other than that it defines a `log` method.

```
class LogManager
  def generate_log: (_Loggable) -> String
end
```

RBS provides the & operator as an intersection operator, so you can define the intersection of, say, multiple interfaces.

If you've used other type systems, like TypeScript or Java, you may be familiar with the concept of a *generic type*. For example, in Ruby an array works basically the same whether it is an array of strings, an array of numbers, or an array of arbitrary user objects. For many purposes, which type makes up the array doesn't matter (for example, the `size` method